Transformer architecture

Main Components of transformer

Embeddings layer

Positional Encoding

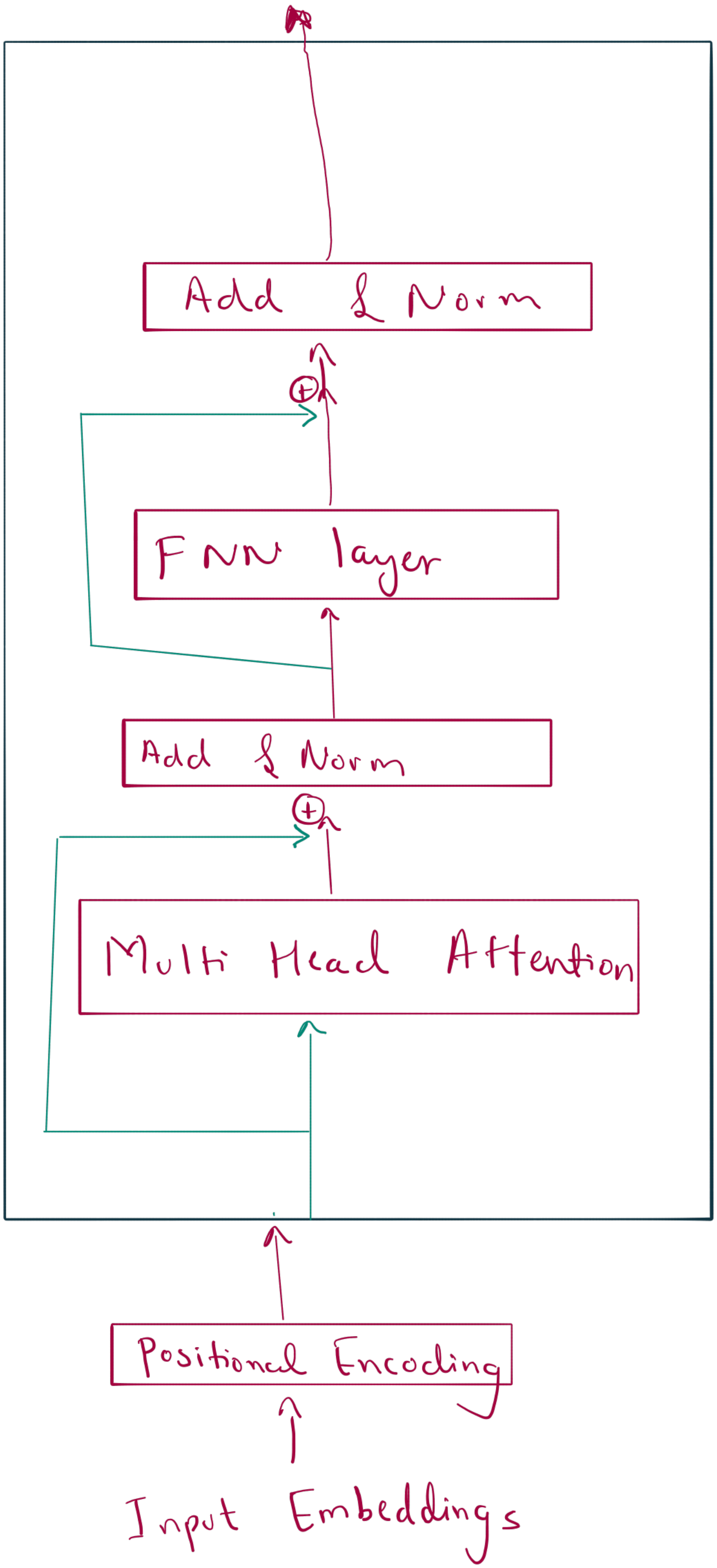
Encoder :- Multi Head attention
- Add and Norm layer
  - Residual Networks
- FNN layer

Decoder :- Masked Multi head attention
- Add and Norm layer
  - Residual layer
- Cross Attention block
- FNN

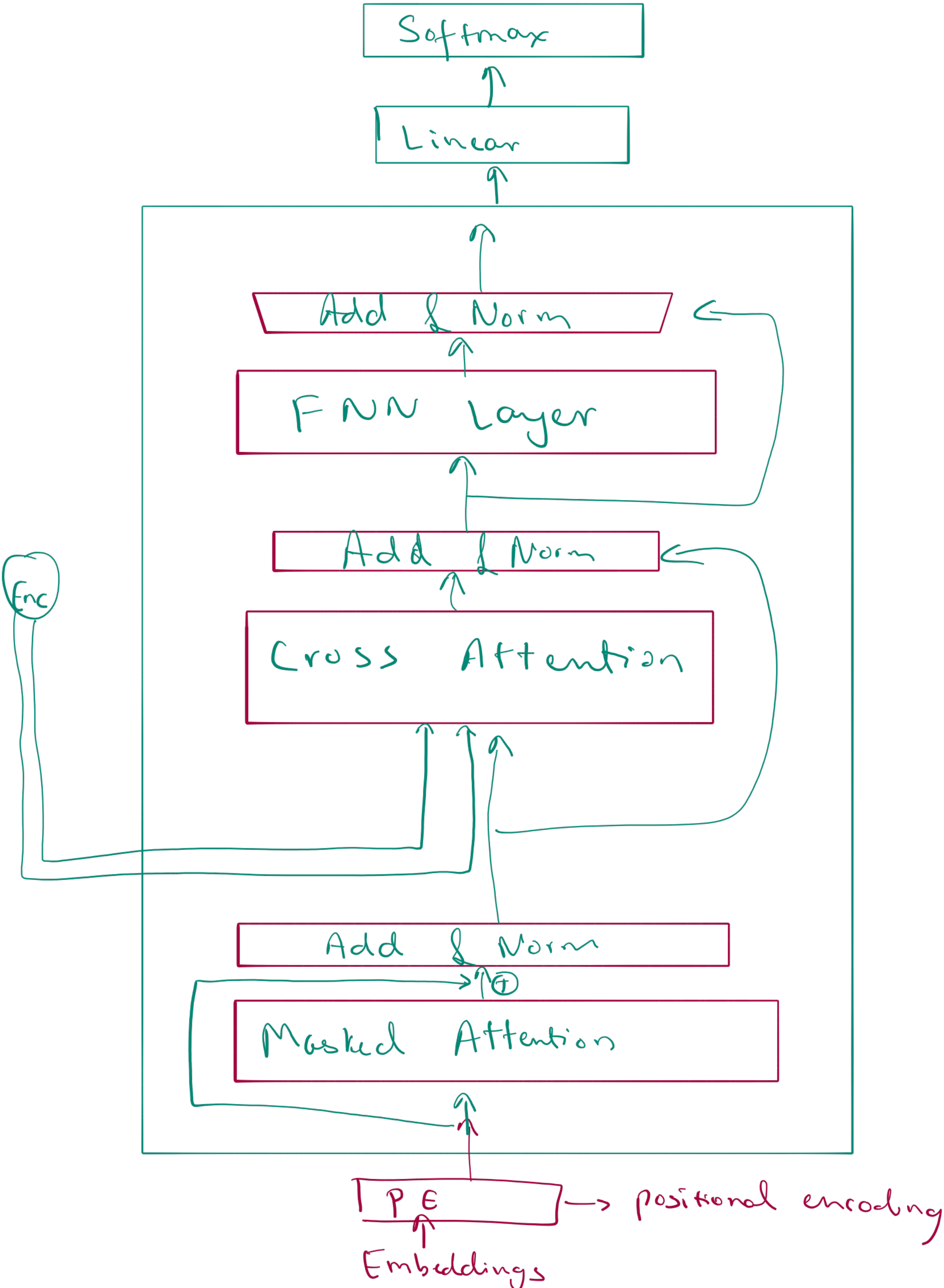
Linear layer

Softmax layer

Encoder Architecture



Decoder Architecture



Encoder

Sentence → Hi, i am vraj → [1, 2, 3, 4]

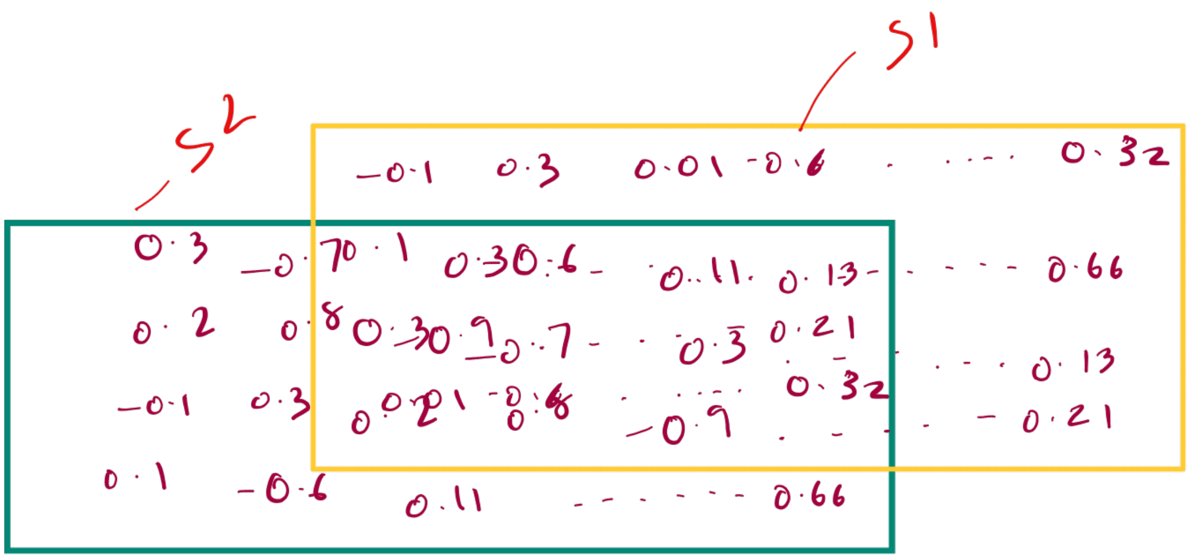
Hello, are you alex → [5, 6, 7, 8]

Sentence 1

		d_1	d_2	d_3	d_4	....	d_{512}
hii :	1	-0.1	0.3	0.01	-0.6	....	0.32
i :	2	0.1	-0.6	0.11		-------	0.66
am :	3	0.3	-0.7	0.3		....	0.13
vraj :	4	0.2	0.8	-0.9		.....	-0.21

Sentence 2

		d_1	d_2	d_3	d_4	....	d_{512}
hello :	1	0.3	-0.7	0.3		....	0.13
are :	2	0.2	0.8	-0.9		.....	-0.21
you :	3	-0.1	0.3	0.01	-0.6	....	0.32
Alex :	4	0.1	-0.6	0.11		-------	0.66



batch size = 2

seq_len = 4

d_model = 512
(embedding)

Shape of tensor : (2, 4, 512)

Embeddings shape (2, 4, 512)

$\downarrow$ e

positional encoding

Zero tensor $\longrightarrow$ (4, 512) shape

Apply

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/dmodel}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/dmodel}}\right)$$

e + PE tensor $\longrightarrow$ final PE tensor

(2, 4, 512)

Final Positional embedded tensor

↓ t

Multi head attention

Needs num_heads to split into heads = 8

d_model = 512

seq_len = 4

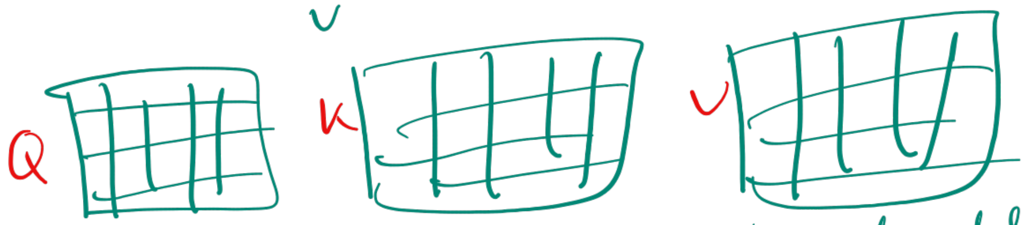
batch_size = 2

initializing d_model _ d_model matrix

w " "

k " "

v



(2, 4, 512) → batch_size, seq len, d_model

↓ 8 heads

(2, 4, 8, 64)

↓

2, 8, 4, 64 — each head sees 64 dims

↓

Multi Head Attention

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Returns 2, 8, 4, 64

|

2, 4, 8, 64

↓

2, 4, 512

Multi head attention output

MHAO

↓

MHAO + E

↓

Final Attention

↓

layer Norm

Requires the tensor x

x = 2, 4, 512

mean →

std →

alpha → ones tensor { parameter

bias → zeros tensor) parameter
ϵ →

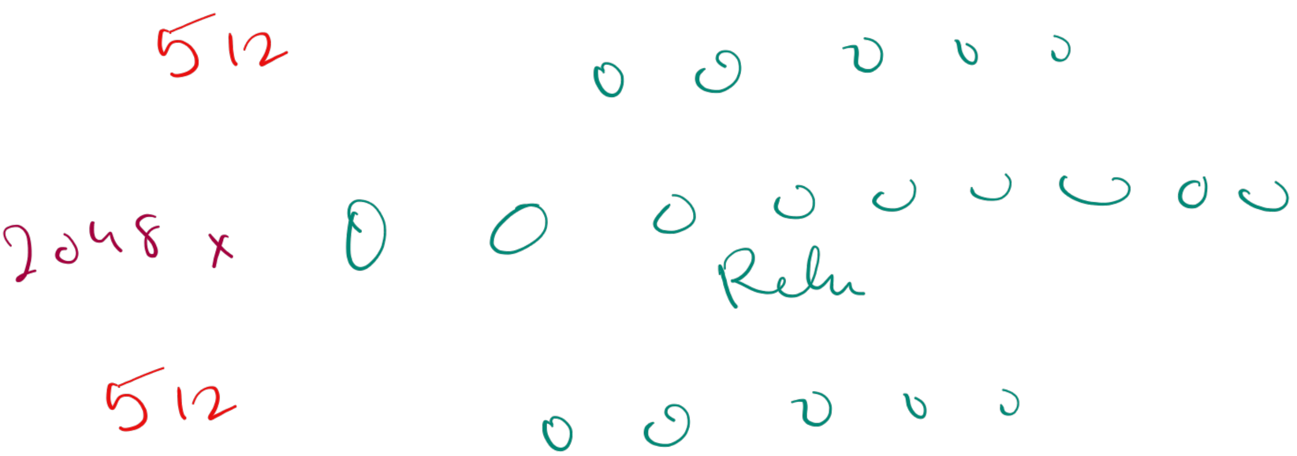
$$\underbrace{\alpha}_{\text{scalar}}\left(\frac{\underbrace{x}_{\text{tensor}} - \overbrace{\text{mean}}^{\text{scalar}}}{\underbrace{\text{std}}_{\text{tensor}} + \underbrace{\epsilon}_{\text{scalar}}}\right) + \text{bias} - \text{scalar}$$

return 2, 4, 512

$\boxed{\text{output}}$ → 2, 4, 512 (B)

↓

$\boxed{\text{FNN}}$



$\boxed{\text{linear output}}$

↓

$\boxed{2, 4, 512}$ output x

output x + B

Add

↓

XB

↓

Norm

↓

2, 4, 512

Repeat X 6 Times

↓

2, 4, 512 → Final encoder output

Decoder Now!

Output sentences

S1 = Hallo, ich bin Vraj

S2 = Hallo, bist du Alex ist?

Padding is applied on Batch level.

Tokenize

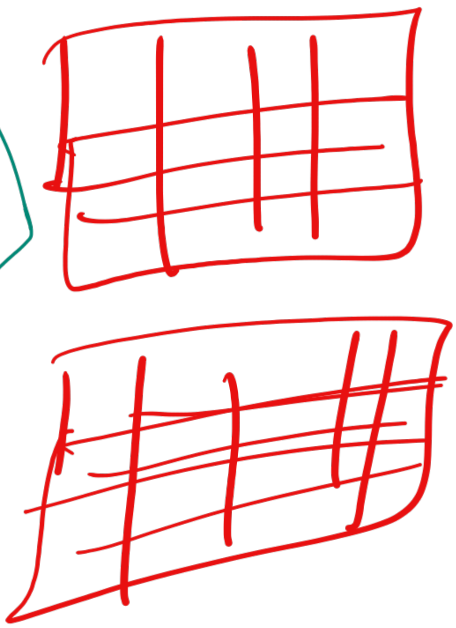
Padding

[1 2 3 4 0]

[1 6 7 8 9]

Embeddings

Note: Padding token also has its own embedding



s_1

5×512

s_2 - 5×512

$(2, 5, 512)$

longest seq len

Masked Multi head Attention

2, 5, 512

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

if mask:
att_sco. Masked_fill_(mask = = 0, -1e9)

att_score softmax (dim = -1)

goes through same Q K V process

2, 5, 512

↓

Add & Norm

↓

2, 5, 512

Cross Attention

Same, just K, V → encoder

Q → decoder

↓

Add & Norm

↓

FNN

↓

Add & Norm

↓

2, 3, 512

↓

Repeat x5 times

↓

2, 5, 512

↓

linear layer

512 → vocab_size = 10

(target_vocab)

everything projects to each word in data.

a

↓

Softmax

→ gives 10 probability

↓

argmax

(picks the winner)

↓

$\boxed{\text{Answer}}$